



Вселенная

Практическое машинное обучение с использованием Scikit-Learn, Keras и TensorFlow. 3-е издание

Автор: Орельен Жерон

Тип файла: pdf

Размер: 26,2 МБ

Язык: Английский

Страниц: 861

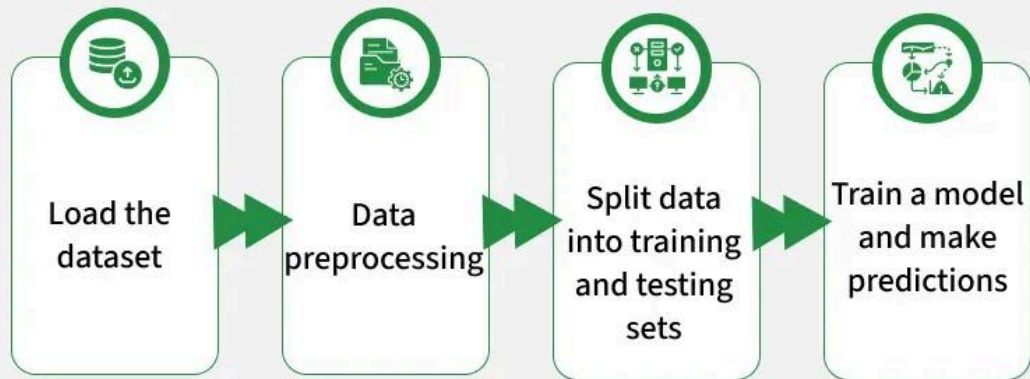
Практическое руководство по машинному обучению с использованием Scikit-Learn, Keras и TensorFlow, 3-е издание: практическое руководство для инженеров

Введение

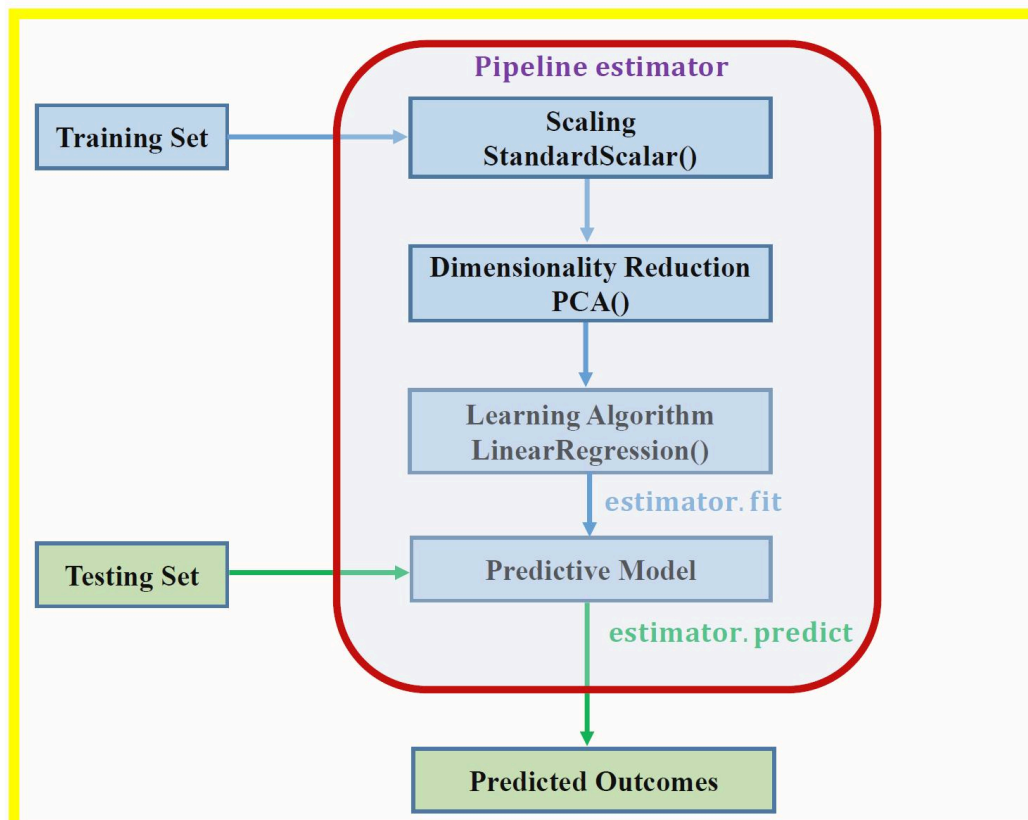
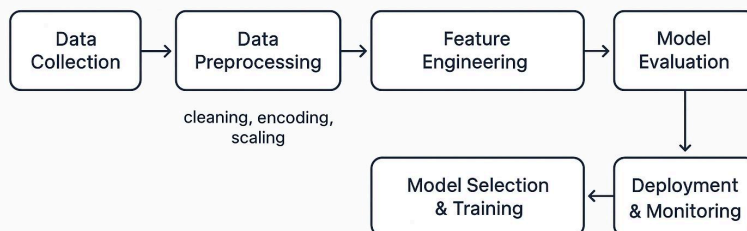
Машинное обучение превратилось из экспериментальной области исследований в практическую инженерную технологию, используемую в разработке программного обеспечения, производстве, здравоохранении, финансах, транспорте, энергетике, робототехнике и многих других отраслях. Однако для создания полезной системы машинного обучения недостаточно выбрать алгоритм и написать код на Python.

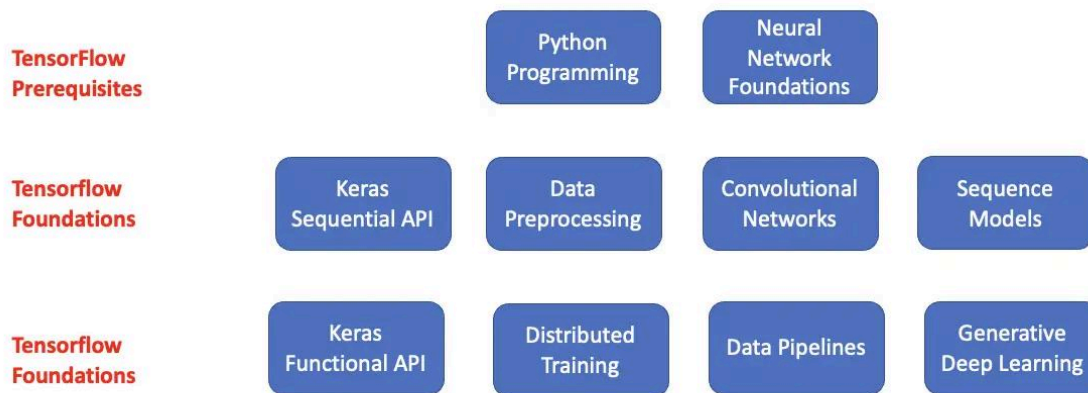
Успешный рабочий процесс обычно включает в себя **подготовку данных, разработку признаков, выбор модели, обучение, валидацию, оценку, развертывание и мониторинг**. Такие инструменты, как **Scikit-Learn, Keras и TensorFlow**, делают эти этапы доступными как для новичков, так и для опытных инженеров.

Steps to Build a Model with Scikit-learn



Machine Learning Workflow with scikit-learn





Scikit-Learn особенно полезен для традиционных алгоритмов машинного обучения и структурированных наборов данных, в то время как Keras предоставляет удобный для разработчиков интерфейс для создания нейронных сетей, а TensorFlow — мощную вычислительную экосистему.

В этой статье мы рассмотрим эти технологии с практической инженерной точки зрения. Вместо того чтобы рассматривать машинное обучение как набор изолированных алгоритмов, мы будем изучать его как **сквозной инженерный процесс**.

Независимо от того, являетесь ли вы студентом, создающим свой первый классификатор, или профессионалом, разрабатывающим интеллектуальную производственную систему, понимание этого процесса может значительно улучшить ваши результаты. 🧠 ⚙️

Теоретическая база

Машинное обучение как инженерный процесс

Традиционная разработка программного обеспечения обычно начинается с четких правил, написанных программистами. Машинное обучение использует другой подход.

Вместо того чтобы вручную прописывать каждое правило, инженеры предоставляют примеры и позволяют алгоритму выявлять полезные закономерности на их основе.

Например, обычная программа может содержать правила для определения вероятности выхода оборудования из строя. Система машинного обучения может выявлять закономерности на основе данных с датчиков, записей о техническом обслуживании, условий эксплуатации и предыдущих сбоев.

Весь процесс можно представить следующим образом:

Данные → Подготовка → Обучение → Оценка → Прогнозирование → Мониторинг

Качество каждого этапа влияет на итоговую систему.

Три важные технологии

Scikit-Learn

Scikit-Learn — это библиотека машинного обучения на Python с единообразным API. Она предоставляет инструменты для:

- Классификация
- Регрессия
- Кластеризация
- Снижение размерности
- Предварительная обработка данных
- Выбор модели
- Перекрестная проверка
- Разработка признаков
- Оценка эффективности
- Конвейеры

Благодаря единообразному интерфейсу он особенно хорошо подходит для изучения основ машинного обучения.

Keras

Keras предлагает высокоуровневый подход к построению нейронных сетей. Инженеры могут создавать модели с помощью слоев, настраивать параметры обучения, подгонять модели под данные, оценивать их и генерировать прогнозы.

Акцент на понятном построении моделей значительно упрощает эксперименты с нейронными сетями.

TensorFlow

TensorFlow — это более широкая платформа для машинного обучения, которая поддерживает численные вычисления, обучение моделей, развертывание, ускорение и производственные процессы.

Keras можно использовать в качестве высокоуровневого интерфейса для моделирования в экосистеме TensorFlow.

Вместе эти технологии могут составить практичный набор инструментов для машинного обучения. 🛠️

Определение

Что такое практическое машинное обучение?

Практическое машинное обучение — это изучение концепций машинного обучения путем внедрения, тестирования, оценки и улучшения реальных моделей, а не только теоретического изучения алгоритмов.

Практический подход обычно включает в себя:

1. Определение проблемы.
2. Сбор данных.
3. Изучение набора данных.
4. Очистка и подготовка данных.
5. Выбор полезных признаков.
6. Разделение набора данных.
7. Обучение базовой модели.
8. Оценка эффективности.
9. Улучшение модели.
10. Развертывание и мониторинг.

The important engineering principle is that **model accuracy is only one part of the solution**.

A highly accurate model may still be unsuitable if it is too slow, expensive, difficult to maintain, biased, or impossible to integrate into an existing system.

Scikit-Learn vs Keras vs TensorFlow

These technologies should not necessarily be viewed as competitors.

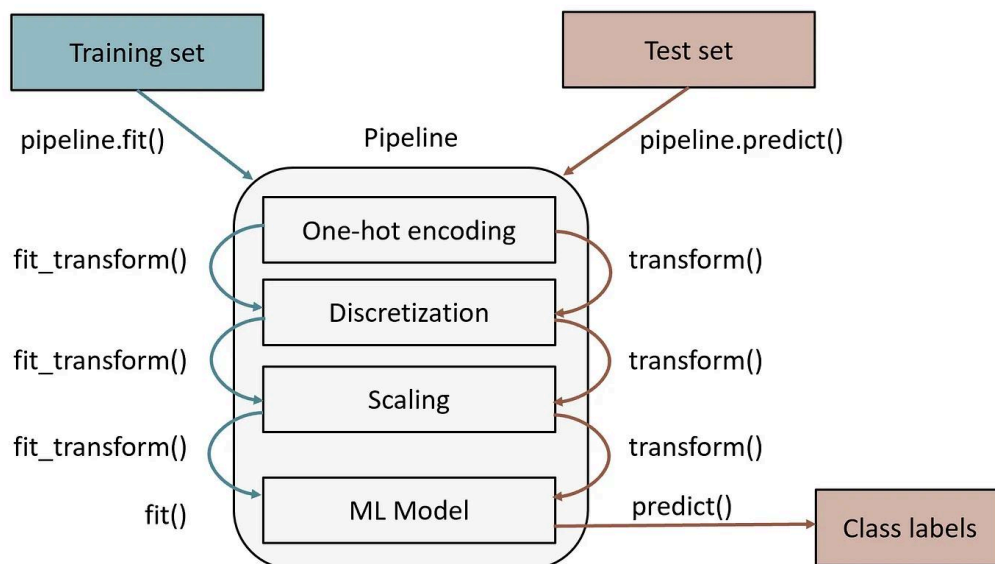
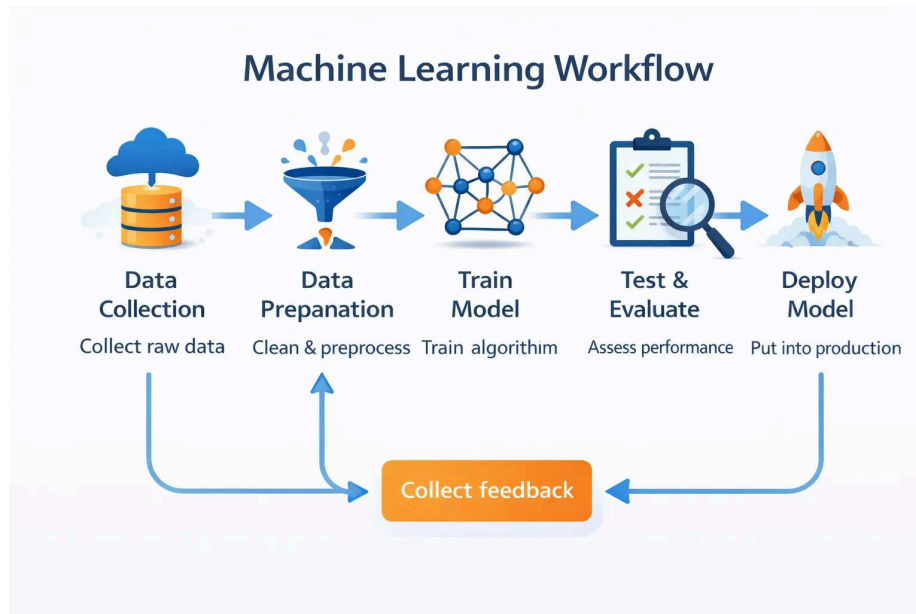
Scikit-Learn is generally excellent for traditional machine learning.

Keras is highly convenient for neural-network development.

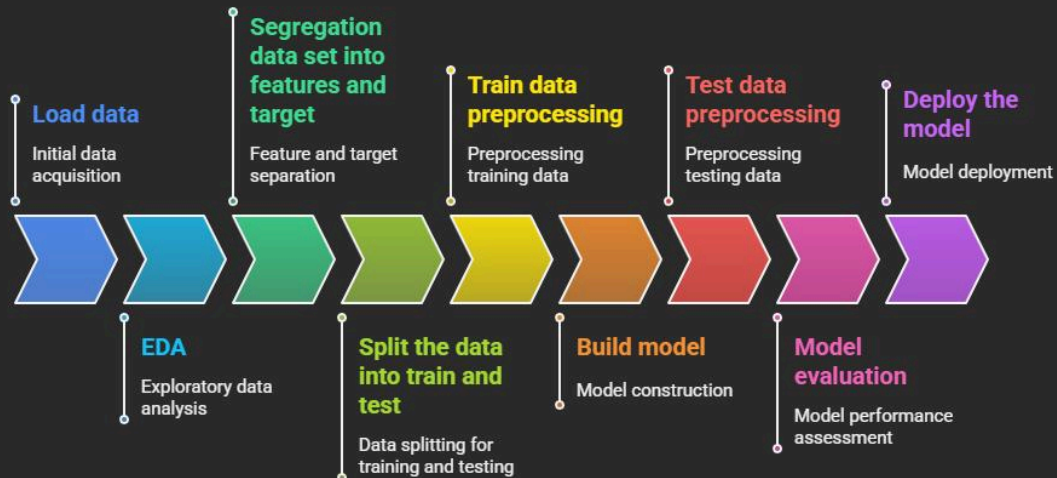
TensorFlow provides a larger ecosystem for building and operationalizing machine-learning systems.

The appropriate technology depends on the problem, dataset, computational requirements, and deployment environment.

Step-by-Step Machine Learning Workflow

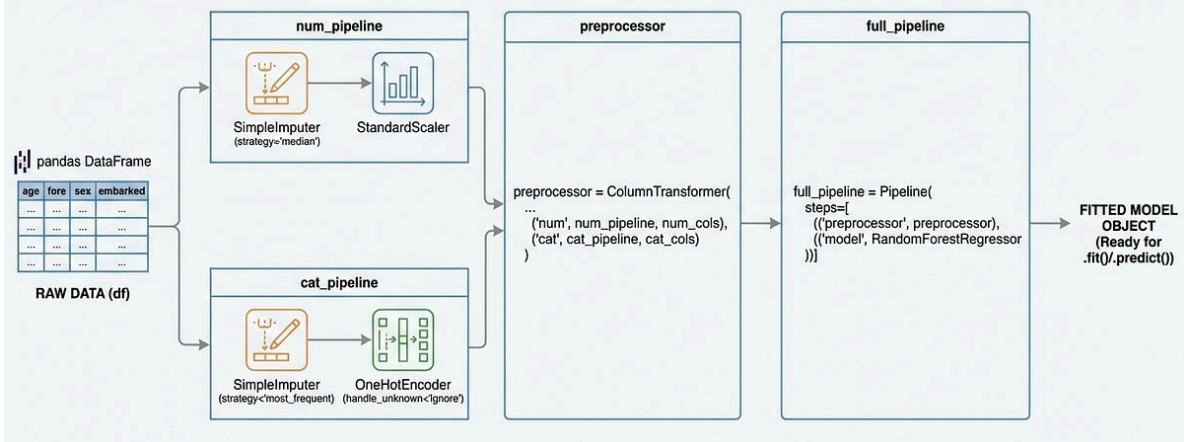


Machine Learning Pipeline: From Data to Deployment



Made with Napkin

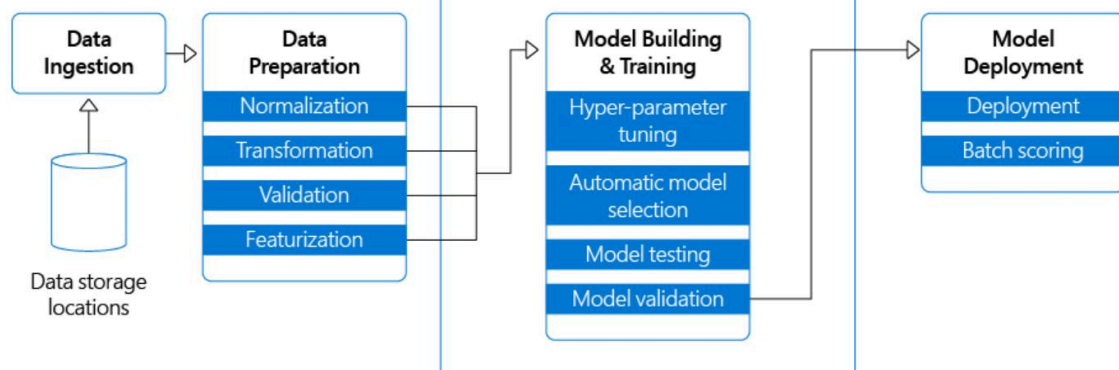
Machine Learning Pipeline

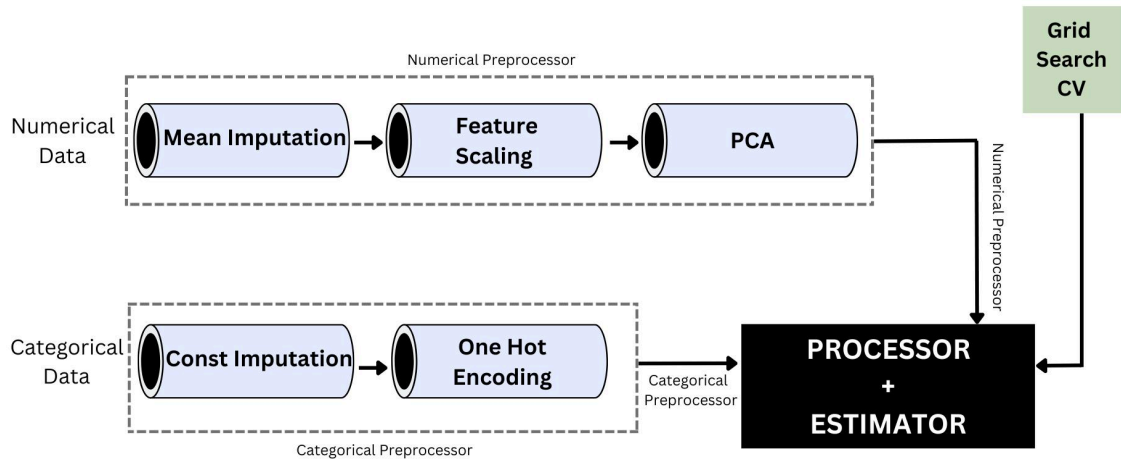


Prepare Data

Build & Train Models

Deploy & Predict





Step 1: Define the Engineering Problem

Before writing code, clearly identify what the system must accomplish.

Ask:

- What information is available?
- What should the system predict?
- Who will use the prediction?
- What happens when the prediction is wrong?
- How will success be measured?

For example, “build an AI model” is not a useful engineering requirement.

“Predict whether a manufacturing component requires maintenance within the next operating cycle” is much more specific.

Step 2: Collect and Understand Data

Machine learning depends heavily on data quality.

Data may originate from:

- Databases
- Sensors
- Web applications
- Transaction systems
- Images
- Text

- Logs
- IoT devices
- Scientific instruments

Before training a model, engineers should investigate missing values, unusual records, duplicated observations, inconsistent units, incorrect labels, and unusual distributions.

Step 3: Prepare the Dataset

Raw information rarely enters a model directly.

Typical preparation tasks include:

- Handling missing values
- Encoding categorical information
- Scaling numerical features
- Removing irrelevant columns
- Correcting inconsistent records
- Creating useful derived features

Step 4: Separate Training and Testing Data

A model should not be judged using exactly the information it learned from.

The dataset is therefore commonly separated into training and testing portions.

The training data is used for learning, while the test data provides an independent assessment.

This simple discipline helps engineers detect models that appear excellent during training but perform poorly on unseen data.

Step 5: Build a Baseline

Do not immediately create the most complicated model possible.

Start with a baseline.

For structured data, this could be a simple linear model, decision tree, logistic classifier, or another suitable Scikit-Learn estimator.

A baseline provides an important reference point.

If a sophisticated neural network performs only marginally better than a simple model, the added complexity may not be justified.

Step 6: Train the Model

In Scikit-Learn, many models follow a simple pattern:

```
1 | model.fit(X_train, y_train)
```

The exact algorithm changes, but the overall workflow remains familiar.

With Keras, neural networks are typically constructed from layers, configured for training, and fitted to data.

The important concept is not memorizing individual commands. It is understanding what each stage accomplishes.

Step 7: Evaluate Performance

Different problems require different evaluation strategies.

For classification, engineers may examine:

- Accuracy
- Precision
- Recall
- F1 score
- Confusion matrix
- ROC-related measures

For regression, useful measures may include:

- Mean absolute error
- Mean squared error
- Root mean squared error
- Coefficient of determination

The correct metric depends on the consequences of errors.

Step 8: Improve and Validate

Once a baseline works, engineers can experiment with:

- Feature selection
- Alternative algorithms
- Hyperparameter tuning
- Regularization

- Different neural-network architectures
- Better preprocessing
- Cross-validation

Step 9: Deploy

A trained model has limited value if nobody can use it.

Deployment might involve:

- A web API
- A cloud service
- An embedded device
- A mobile application
- A manufacturing controller
- A data-processing pipeline

Step 10: Monitor

Deployment is not the end.

Real-world data changes.

Customer behavior changes. Sensors degrade. Markets evolve. Software systems are modified.

Therefore, production machine-learning systems should be monitored for performance degradation, data drift, failures, latency, and unexpected predictions. 📊

Comparison

Scikit-Learn vs Keras vs TensorFlow

Feature	Scikit-Learn	Keras	TensorFlow
Main focus	Traditional ML	Neural networks	ML platform
Beginner friendliness	Excellent	Excellent	Moderate
Classification	Excellent	Excellent	Excellent
Regression	Excellent	Excellent	Excellent

Feature	Scikit-Learn	Keras	TensorFlow
Deep learning	Limited	Excellent	Excellent
Structured data	Excellent	Excellent	Excellent
Computer vision	Limited	Excellent	Excellent
NLP	Limited	Excellent	Excellent
Model pipelines	Excellent	Good	Excellent
Rapid experimentation	Excellent	Excellent	Good
Production ecosystem	Strong	Strong	Very strong

When Should You Choose Each?

Choose Scikit-Learn When

You are working with structured tabular data and traditional machine-learning algorithms.

Examples include:

- Customer classification
- Predictive maintenance
- Risk assessment
- Sales forecasting
- Regression
- Clustering

Choose Keras When

You need neural networks and want a relatively readable development experience.

Examples include:

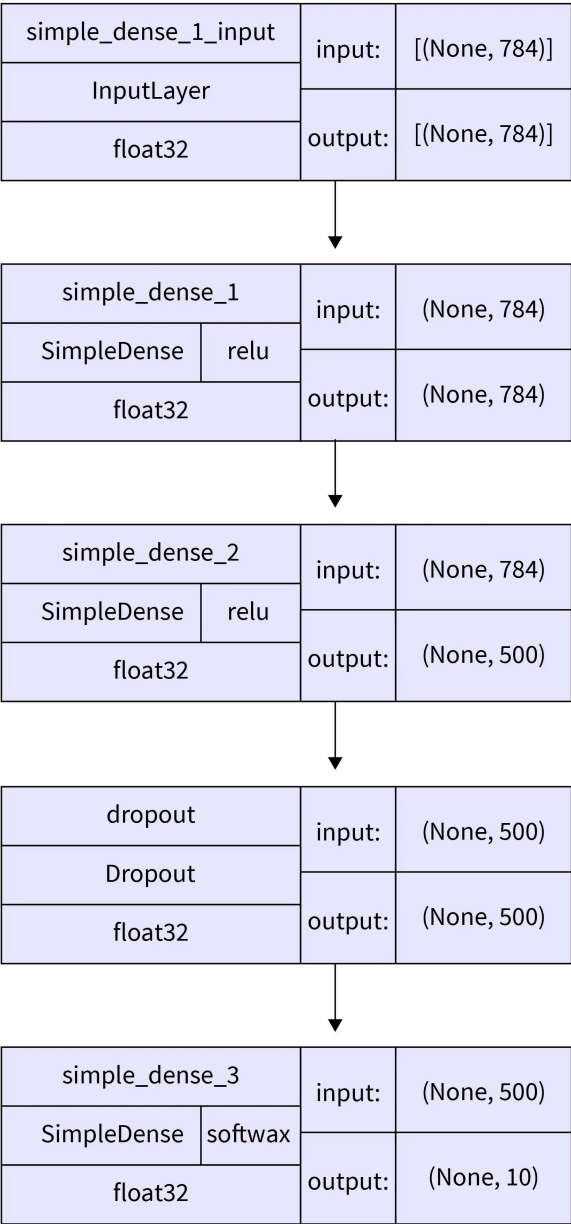
- Image classification
- Text classification
- Sequence modeling
- Deep neural networks

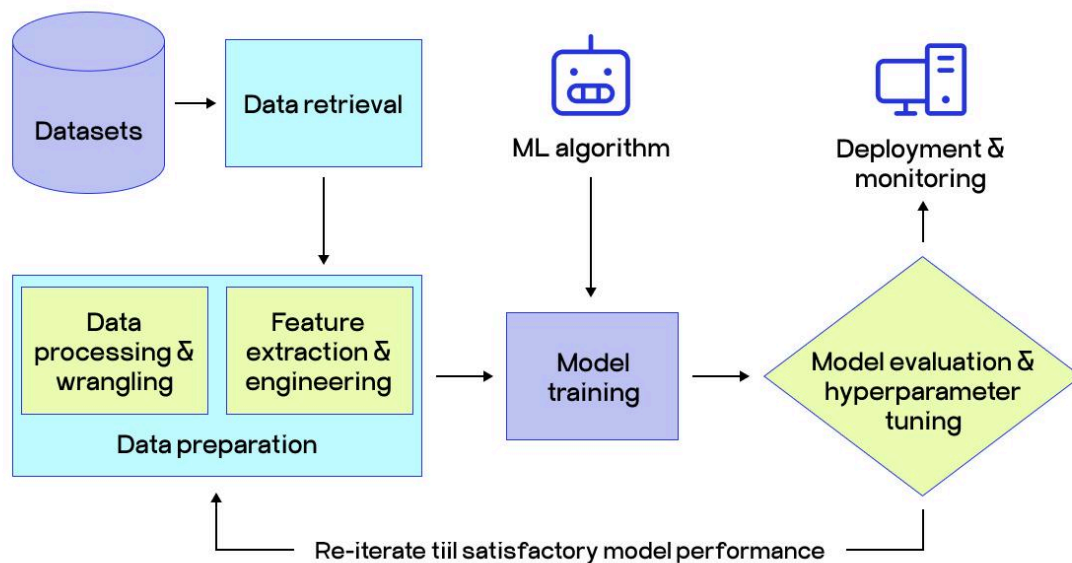
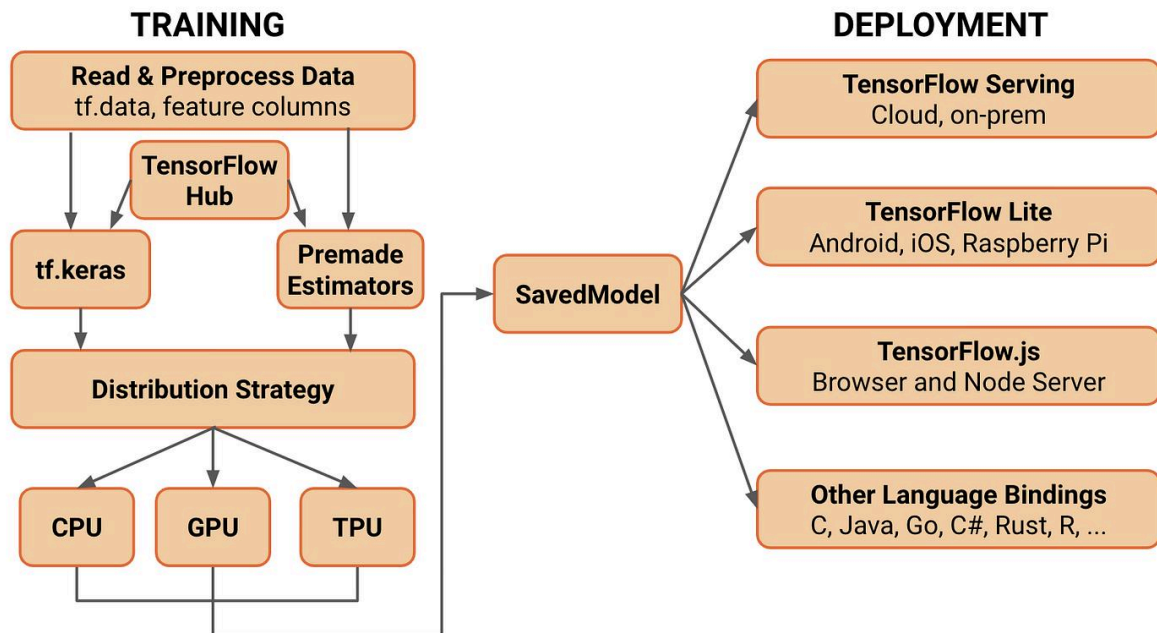
Choose TensorFlow When

You need broader machine-learning infrastructure, specialized computation, or production-oriented deployment capabilities.

In many projects, these tools can complement rather than replace each other.

Diagrams & Tables





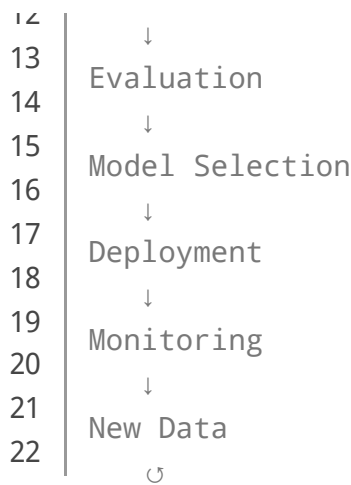
A Practical Architecture

A simplified engineering architecture looks like this:

```

1 | Raw Data
2 |   ↓
3 | Data Validation
4 |   ↓
5 | Preprocessing
6 |   ↓
7 | Feature Engineering
8 |   ↓
9 | Train / Validation / Test
10|   ↓
11| Model Training
12|

```



The important idea is the feedback loop.

A production model should generate information that helps engineers determine whether it continues to perform correctly.

Examples

Example 1: Predictive Maintenance

Imagine a factory containing hundreds of industrial motors.

Sensors continuously record:

- Temperature
- Vibration
- Operating hours
- Load
- Rotation behavior

A Scikit-Learn model could learn relationships between these measurements and previous maintenance events.

The final system might classify equipment into categories such as:

Normal → Attention Required → High Maintenance Risk

Engineers could then schedule inspections before catastrophic failure.

Example 2: Image Classification

Suppose an engineering company needs to identify defects in manufactured components.

A camera captures images of each component.

A Keras neural network can learn visual patterns associated with acceptable and defective components.

The model can then classify new images automatically.

This approach could support automated quality-control systems.

Example 3: Energy Forecasting

A building-management system could collect historical energy consumption, weather information, occupancy patterns, and operating schedules.

A machine-learning model could estimate future energy demand.

Facility managers could use these predictions to optimize HVAC systems and reduce unnecessary consumption. ⚡

Real-World Applications

Manufacturing

Machine learning can support:

- Predictive maintenance
- Automated inspection
- Process optimization
- Fault detection
- Production forecasting

Civil Engineering

Possible applications include:

- Structural condition assessment
- Construction productivity prediction
- Material-property estimation
- Project risk analysis
- Traffic forecasting

Electrical Engineering

Machine-learning systems can help with:

- Load forecasting
- Fault detection
- Renewable-energy prediction
- Power-quality analysis
- Smart-grid optimization

Mechanical Engineering

Applications include:

- Equipment diagnostics
- Remaining-useful-life prediction
- Design optimization
- Vibration analysis
- Robotics

Software Engineering

Machine learning is increasingly applied to:

- Anomaly detection
- Recommendation systems
- Intelligent search
- Fraud detection
- Automated classification

Common Mistakes

Training on Poor Data

A sophisticated algorithm cannot compensate for unreliable data.

Solution: validate the dataset before investing heavily in model development.

Data Leakage

Data leakage occurs when information that should be unavailable during prediction accidentally influences training.

Solution: use disciplined train/test separation and preprocessing pipelines.

Optimizing Only for Accuracy

Accuracy may be misleading for imbalanced classification problems.

Solution: select metrics according to the engineering consequences of errors.

Using an Overly Complex Model

A deep neural network is not automatically better than a simpler algorithm.

Solution: establish a baseline and justify additional complexity.

Ignoring Deployment

A model that works inside a notebook may fail in production.

Solution: consider memory, latency, dependencies, security, monitoring, and integration requirements from the beginning.

Challenges & Solutions

Challenge	Engineering Solution
Missing data	Imputation and validation
Imbalanced classes	Appropriate sampling and metrics
Overfitting	Validation, regularization, and simpler models
Large datasets	Efficient pipelines and scalable infrastructure
Model drift	Continuous monitoring
High latency	Model optimization and efficient serving
Difficult maintenance	Modular pipelines and version control
Poor interpretability	Explainability techniques and appropriate model selection

Case Study

Intelligent Manufacturing Inspection

Consider a hypothetical manufacturing facility producing precision mechanical components.

The company previously relied on manual inspection. Engineers noticed that defects were sometimes discovered only after components reached later stages of production.

The engineering team designed a machine-learning inspection system.

Phase 1: Data Collection

The team collected images from cameras positioned around the production line.

Each image was labeled according to inspection results.

Phase 2: Data Preparation

Images were checked for quality and organized into appropriate training and evaluation groups.

Poor-quality images were removed or reviewed.

Phase 3: Model Development

Engineers first established a simple baseline.

They then developed a neural-network solution using Keras and TensorFlow because the problem depended heavily on visual patterns.

Phase 4: Evaluation

The team examined more than a single performance number.

They investigated false positives and false negatives separately because missing a defective component could have a much greater operational cost than unnecessarily inspecting an acceptable component.

Phase 5: Deployment

The trained model was connected to the production inspection system.

New component images were automatically analyzed.

Phase 6: Monitoring

Engineers continued tracking prediction quality because lighting conditions, camera positioning, component designs, and production processes could change.

This case demonstrates an important principle:

Machine learning is a system, not merely a model.

Essential Tips

Start Small

Build a simple working system before creating a sophisticated one.

Understand Your Data

Spend significant time investigating the dataset.

Use Reproducible Pipelines

Automate preprocessing and model training wherever practical.

Keep the Test Set Honest

Do not repeatedly optimize directly against your final test set.

Track Experiments

Record:

- Dataset version
- Features
- Model type
- Hyperparameters
- Metrics
- Training configuration

Think About the User

The best model is not always the one with the highest benchmark score.

Consider usability, cost, speed, reliability, interpretability, and maintenance.

Learn the Workflow, Not Just the Library

Libraries evolve.

APIs change.

Algorithms are replaced.

But the fundamental workflow—**problem definition** → **data** → **preparation** → **training** → **evaluation** → **deployment** → **monitoring**—remains highly valuable.

FAQs

What is [Scikit-Learn](#) mainly used for?

Scikit-Learn is widely used for traditional machine-learning tasks such as classification, regression, clustering, preprocessing, model selection, and evaluation.

Is Keras suitable for beginners?

Yes. Keras provides a relatively accessible interface for constructing and training neural networks, making it a popular starting point for deep-learning education.

Is TensorFlow the same as Keras?

No. They are related technologies, but they are not identical. Keras provides a high-level modeling interface, while TensorFlow provides a broader machine-learning and computational ecosystem.

Should I learn Scikit-Learn before TensorFlow?

For many beginners, learning Scikit-Learn first is useful because it teaches fundamental concepts such as training, testing, preprocessing, feature engineering, validation, and evaluation.

However, learners focused specifically on deep learning can begin with Keras.

Can Scikit-Learn and TensorFlow be used together?

Yes. Different components of a machine-learning system can use different tools depending on their requirements.

Do I need advanced mathematics?

A basic understanding of statistics, probability, vectors, and optimization is valuable. However, beginners can initially build practical models without mastering every mathematical detail.

As your engineering responsibilities increase, deeper mathematical understanding becomes increasingly useful.

Is machine learning useful for engineers?

Absolutely. Machine learning can assist engineers in prediction, anomaly detection, optimization, automation, quality control, diagnostics, and decision support.

Which technology should I learn first?

A practical sequence is:

Python → NumPy/Pandas → Scikit-Learn → machine-learning fundamentals → Keras → TensorFlow → deployment and MLOps.

This sequence gives beginners a strong foundation while leaving room for advanced specialization.

Conclusion

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow represents more than a collection of programming techniques. It provides a practical pathway for transforming raw information into useful engineering systems.

Scikit-Learn is an excellent environment for learning traditional machine learning and building structured-data solutions. Keras simplifies neural-network development, while TensorFlow provides a broader ecosystem for advanced machine-learning workflows.

The most important lesson is to avoid thinking of machine learning as simply:

“Choose model → train → get accuracy.”

Professional machine learning is closer to:

Problem → Data → Validation → Preprocessing → Features → Model → Evaluation → Deployment → Monitoring → Improvement 🔄

For students, this workflow provides a strong foundation for projects and research. For professionals, it provides a framework for designing reliable intelligent systems.

The future of engineering will increasingly combine traditional engineering knowledge with data-driven methods. Learning to use Scikit-Learn, Keras, and TensorFlow therefore gives engineers a practical bridge between **classical engineering principles, software development, artificial intelligence, and real-world automation.** 🚀

The diagrams and workflows referenced above are presented as conceptual illustrations; the article text itself is original.

Related Posts:



[Preview PDF Book](#)

[← Previous Book](#)

[Next Book →](#)

Explore a vast library of free engineering ebooks curated to help you expand your technical knowledge, conduct groundbreaking research, and advance your career. Our mission is to create a hub of knowledge that fosters innovation, empowers aspiring engineers, and bridges the gap between education and industry.